
OWASP
GLOBAL AppSec
SF'26
November 2-6

OWASP GLOBAL AppSec SF'26 November 2-6

25 years of open source security

OWASP GLOBAL APPSEC SAN FRANCISCO · NOV 5, 2026

Reproducing the exploit, not the report

Hedi Sfaxi

WHO I AM

Hedi Sfaxi

🇹🇳 → 🇫🇷

Product engineer, konvu

> I build agents that **triage, reproduce and remediate** vulnerabilities

Generation	**Z**
Attention span	**Short**
Skinny jeans	**Never**
AI-pilled	**Yes**

THE PROBLEM

Finding bugs got cheap

AI harnesses scan whole codebases overnight. Coding agents flag bugs in every PR. Bounty hunters file reports in bulk.

Finding is cheap now. Confirming is not.



1 · THE PROBLEM

Finding vulnerabilities got cheap

01

Models are getting smarter

Frontier models now reason across whole codebases.

02

Specialized OSS harnesses are available

Agents scan a repo, then revalidate their own findings.

03

Bug bounty hunters + AI

Experience and expertise, multiplied by agents: a deadly combo.

HARNESSES AVAILABLE TODAY

deepsec
Vercel · open source

VVAH
Visa · open source

Claude Code Security
Anthropic

Codex Security
OpenAI

1 · THE PROBLEM

A typical validation flow

Choose a target, reproduce the claim, then check the effect.

01
Report
Claim and prerequisites

→

02
Triage
Valid, in scope, not a duplicate?

→

03
Lab / staging / production
Matching target and identities

→

04
Reproduce
Access scoped to the claim

→

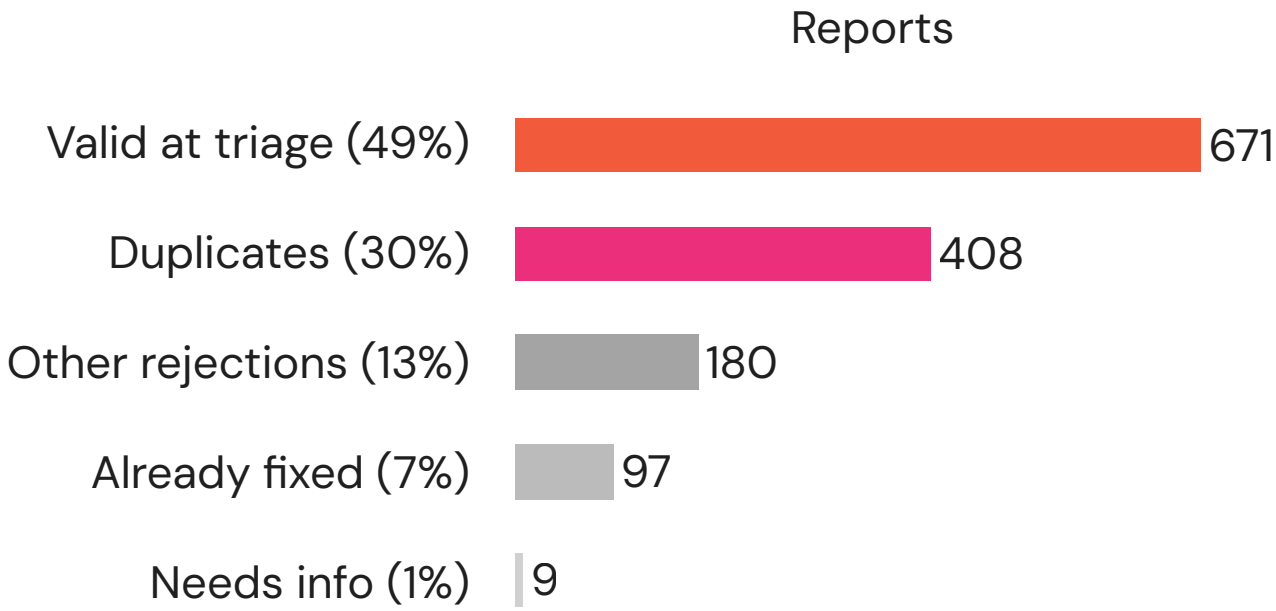
05
Evidence
Target or victim observation

1 · THE PROBLEM

The investigation workload

1,365

reports triaged



671 reports passed triage. Each still needed validation.

AppSec team, bounty inbox, 2026



1 · THE PROBLEM

The claim still needs a working target

THE REPORT

A Viewer can delete another organization's snapshot.

```
DELETE /api/snapshots/{key}
```

THE TARGET WE NEED

Version	**Grafana 10.2.5**
Victim	**Org A's existing snapshot**
Attacker	**Org B, Viewer**
Access	**View key, no delete key**
Measure	**Snapshot state before/after**

1 · THE PROBLEM

Two forces that fight each other

Production — Staging — **Isolated lab**

Production realism

The right version, data, identities and integrations.

Freedom to experiment

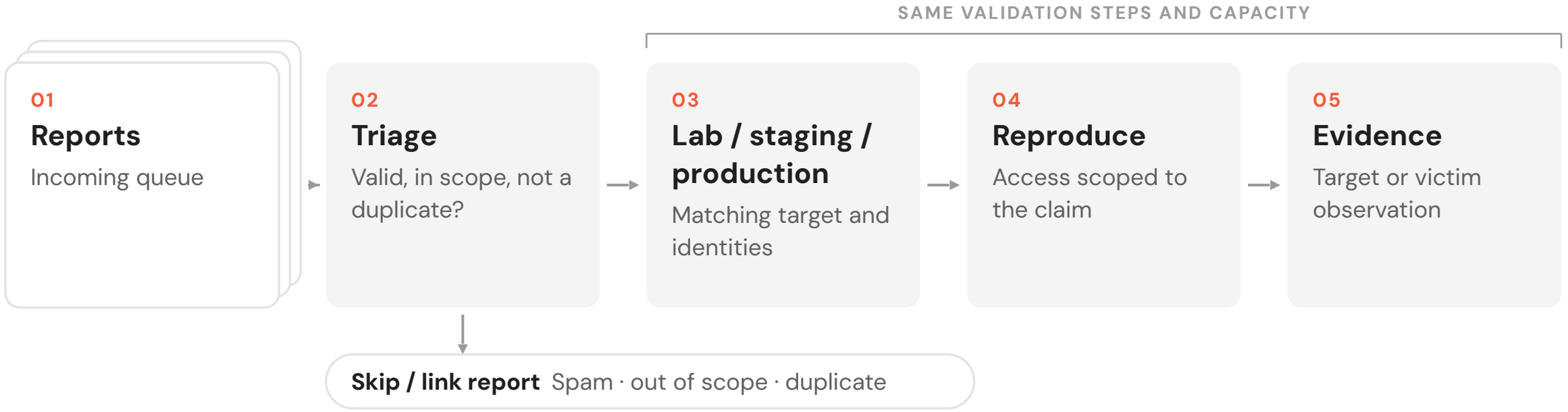
Configure the target, change versions and reproduce.

> Our choice: **rebuild the required conditions in an isolated lab.**

1 · THE PROBLEM

More reports, same validation capacity

More findings still need a target, a reproduction and evidence.



We need to **scale validation.**

Illustrative queue

1 · THE PROBLEM

Recap

01

Finding is cheap

AI harnesses, coding agents and bounty hunters file reports in bulk.

02

Every claim needs a target

The right version, data, identities and integrations.

03

Validation doesn't scale

Same steps, same capacity, more reports.

Next: **rebuild the target, run the attack, inspect the evidence.**

We built it like an assembly line

Rebuild the target. Run the attack. Inspect the evidence.

2 · WHAT WE BUILT

Why an assembly line

01 Predictable workflow
Stable I/Os

02 Chained stations
Contracts and success metrics

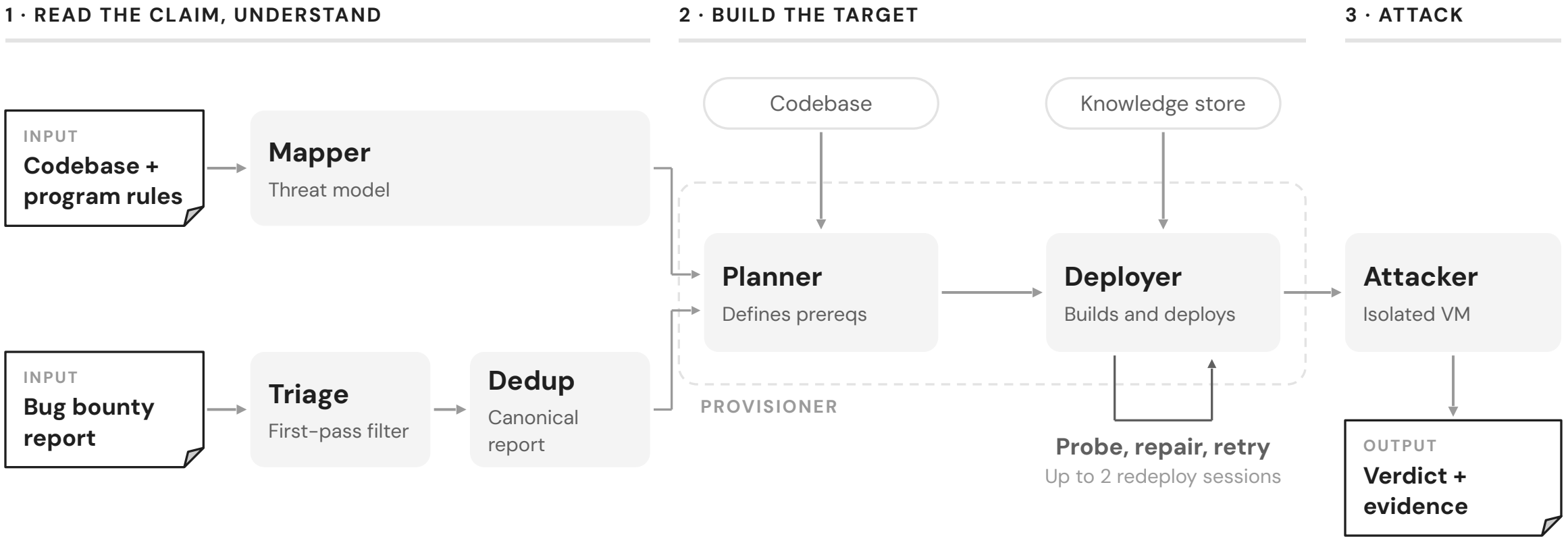
03 Improving workers
More intelligent each month

04 Fixed toolset
Set of tools and constraints



2 · WHAT WE BUILT

How a report becomes a testable target

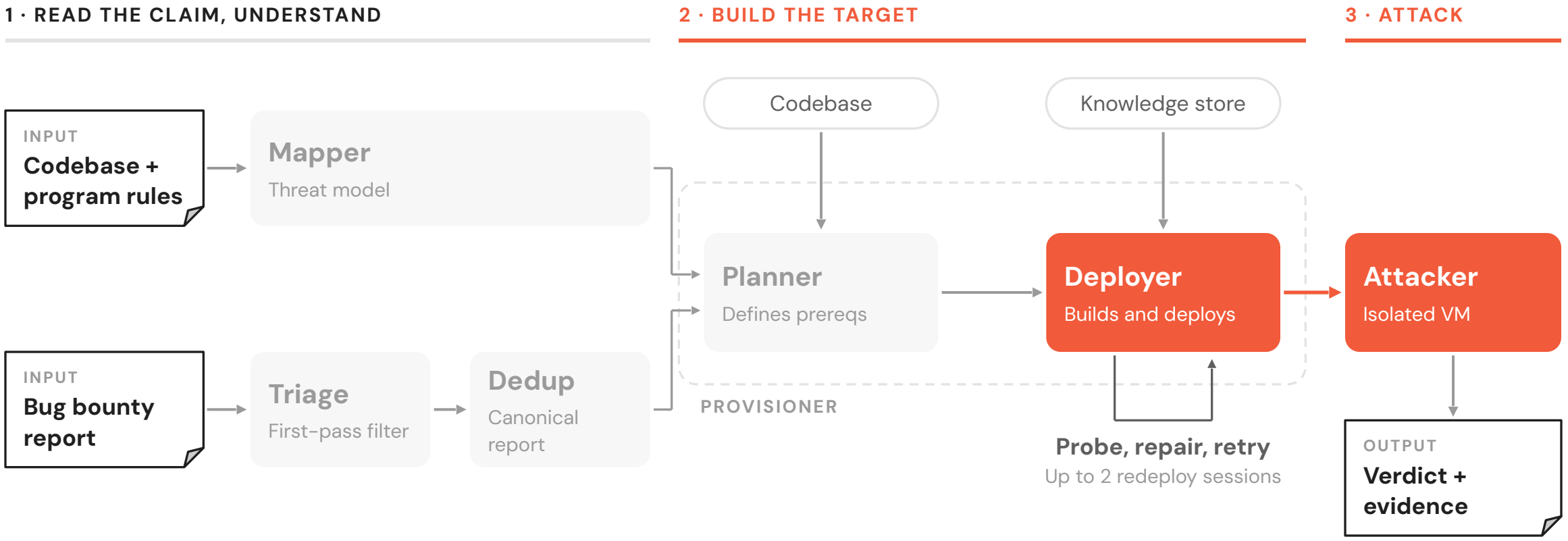


After the attack: evidence inspection, teardown and reporting

2 · WHAT WE BUILT

Typed contracts | Tools for the claim | Focused roles

Zoom: the Deployer–Attacker handoff



After the attack: evidence inspection, teardown and reporting

2 · WHAT WE BUILT

Typed contracts between agents

Deployer
writes target.json

→

Worker
validates the contract

→

Attacker
receives typed inputs

target.json · illustrative Grafana excerpt

```
{
  "app": "grafana", "version": "10.2.5",
  "target": {
    "base_url": "http://10.1.0.12:3000",
    "port": 3000
  },
  "credentials": {"viewer": {"role": "Viewer"}}
}
```

A gate before the next agent

`TargetOutput.model_validate(data)`

`port: 3000` → **Continue**

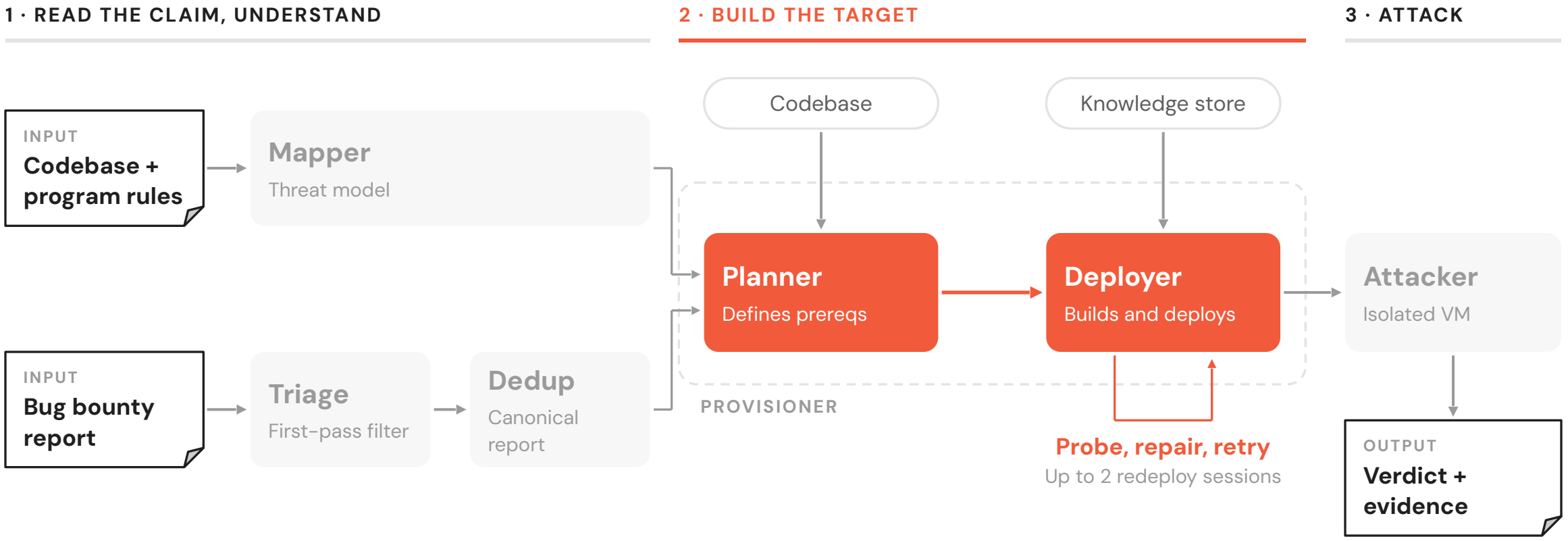
`port: 70000` → **Stop**

Invalid output never reaches the attacker.

This checks the data structure. Live readiness checks come later.

2 · WHAT WE BUILT

Zoom: planning and deployment



After the attack: evidence inspection, teardown and reporting

2 · WHAT WE BUILT

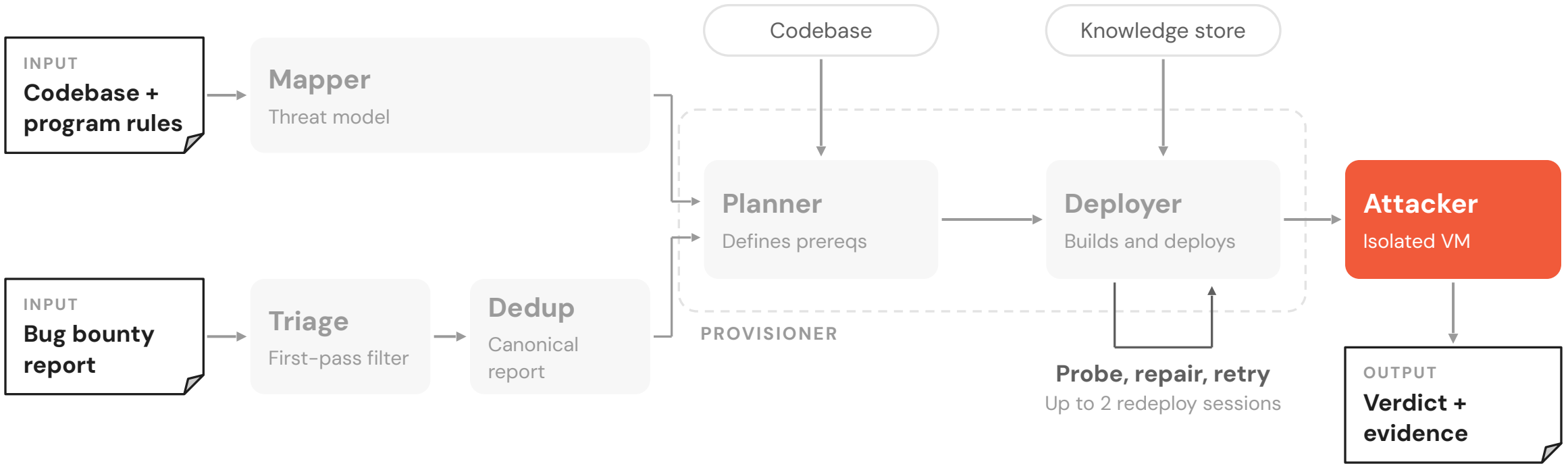
Setup and tools matched to the claim

The stages stay the same. The setup and tools follow the claim.

EXAMPLE REPORT	TARGET SETUP AND INTERACTION TOOLS	
API access control →	**API + database** HTTP requests from two accounts with different access	`curl` `psql`
Stored XSS →	**Web app + browser** Browser actions in a separate victim session	`Chromium`

2 · WHAT WE BUILT

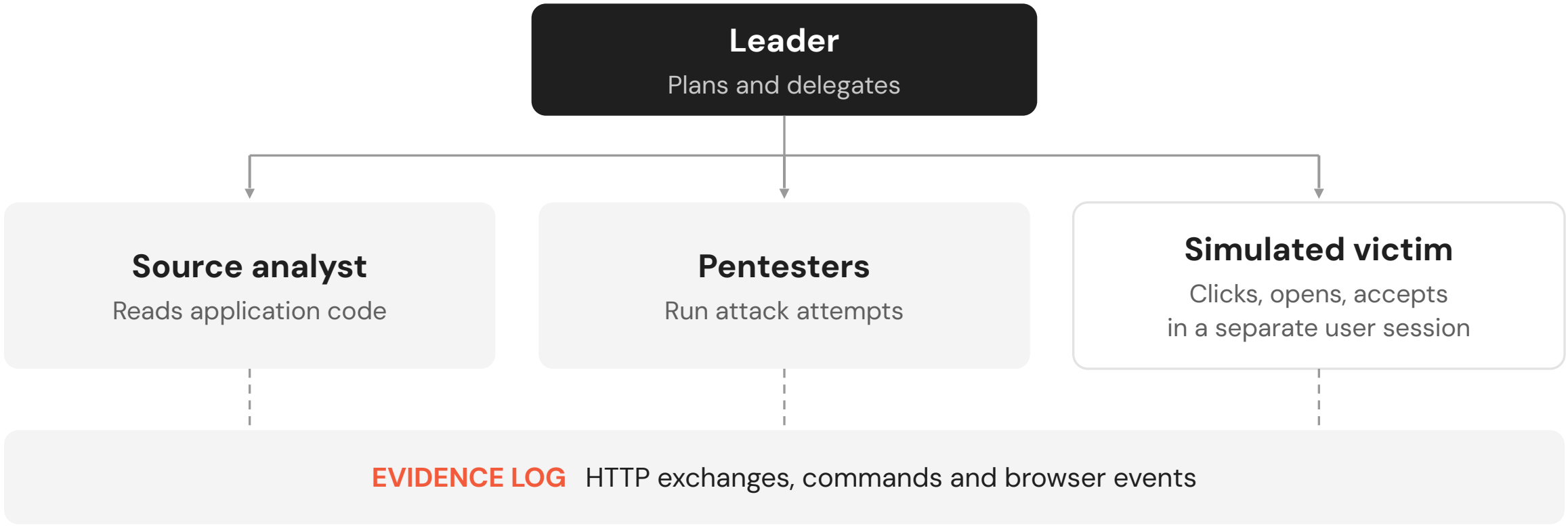
Zoom: inside the attacker stage



After the attack: evidence inspection, teardown and reporting

2 · WHAT WE BUILT

Focused roles inside the attacker stage



2 · WHAT WE BUILT

The run ends with one of three outcomes

OUTCOME	MEANS	EXAMPLE FROM OUR RUNS
Exploitable	Observed impact supports the claim.	Grafana CVE-2024-1313: snapshot deleted across orgs
Not exploitable	Evidence contradicts the claim in this setup.	Grafana image already contained the fix
Inconclusive	The experiment could not settle the claim.	XSS fired, invitation step never tested

A broken lab leaves the claim **inconclusive.**

BUILDING THE TARGET

If the lab fails, the claim stays untested

Check versions, seed data and user roles before spending an attacker run.

3

3 · BUILDING THE TARGET

Where our runs failed

COST

53%

of recorded stage cost

TIME

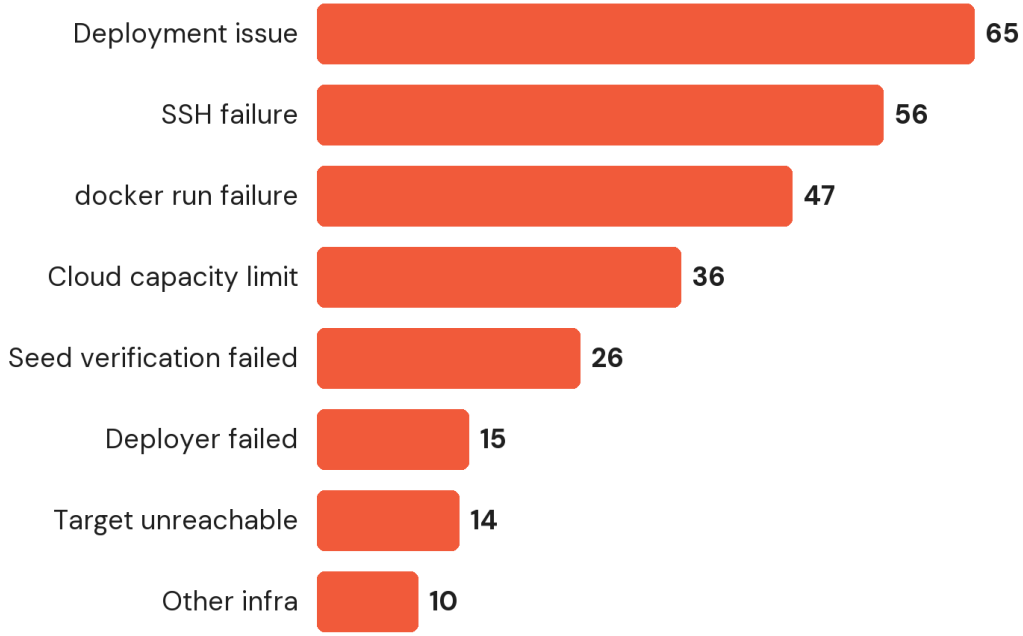
42%

mean share of run time

ERRORS

64%

of inconclusive runs: infra or deployment



Cost and time: deployer stage, where recorded.

3 · BUILDING THE TARGET

Five deployment patterns

Check the build 1

Inspect the deployed build.

Limit repairs 2

Repair within a fixed budget.

Verify user roles 3

Verify the account's actual role.

Preserve state 4

Keep the target intact for the test.

Reuse setup fixes 5

Reuse setup lessons.

3 · BUILDING THE TARGET

1. Check the code in the deployed build

Recorded Grafana case: the requested image already contained the fix.

Recorded source finding

```
auth_jwt.go
stripSensitiveParam()  # present
```

The fix is present.

Stop before the test.

WHAT GOES WRONG	WHAT WE ASK THE AGENT	WHAT THE WORKER ENFORCES
Requested image already contains the fix	Look for the fix in the source, record it in target.json	**Reads the stop signal, never starts the attacker**

3 · BUILDING THE TARGET

2. Give repairs a limit

Feed the failure back to the deployer, then probe again.

Repair loop (condensed)

```
while reason and attempts < 2:
    attempts += 1
    redeploy()
    reason = smoke_test_exploit_endpoint(...)
```

Two repair sessions.

A persistent server error stops the run.

WHAT GOES WRONG

The agent loops on a setup it can't fix

→

WHAT WE ASK THE AGENT

Record the failure reason, then redeploy

→

WHAT THE WORKER ENFORCES

Caps repairs at 2 sessions, then stops the run

3 · BUILDING THE TARGET

3. Verify the account's actual role

A successful login can still leave you testing with the wrong permissions.

Example credential check

```
"planned_role": "Viewer",
"deployed_role": "Admin",
"role_verified": false
```

Expected Viewer. Found Admin.

→ Go one step back and fix the deployment.

WHAT GOES WRONG

Login works, but with the wrong role

→

WHAT WE ASK THE AGENT

Report the planned and the deployed role

→

WHAT THE WORKER ENFORCES

Role mismatch sends the run back to deployment

Deployer: “let me smoke-test the exploit endpoint”

deletes the snapshot

Attacker: “where did the snapshot go?”



3 · BUILDING THE TARGET

4. Keep readiness checks from changing the target

The setup check must leave the snapshot available for the test.

Snapshot deletion example

```
Planned test    DELETE
Setup probe     OPTIONS
```

Check the route.

Leave deletion for the reproduction step.

WHAT GOES WRONG

The smoke test fires the exploit and deletes the data

→

WHAT WE ASK THE AGENT

Probe the route without the destructive call

→

WHAT THE WORKER ENFORCES

Excludes destructive actions during setup

5. Carry setup fixes into the next run

Save deployment lessons as files and check whether they still apply.

Lesson file (structure)

```
deployment-lessons/<id>.md
  Problem
  Solution
  Applicable versions
```

The next planner and deployer read the lessons before starting again.

WHAT GOES WRONG

Each run rediscovers the same setup fix

→

WHAT WE ASK THE AGENT

Write a lesson: problem, solution, versions

→

WHAT THE WORKER ENFORCES

Next planner and deployer read lessons that still apply

Part 3 gave us a lab we can trust. Now: a result we can trust.

CHECKING THE RESULT

Did the target's state change?

A reproduced exploit changes something observable on the target or victim side.

An agent saying "success" is not evidence.

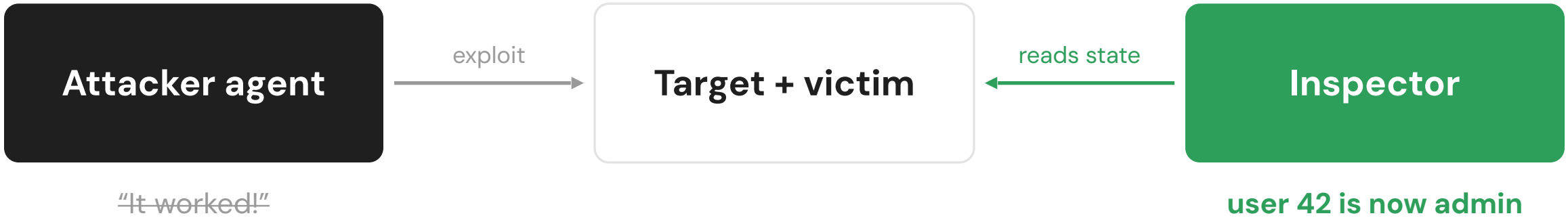
4


Get a bug report
Ask an agent to exploit it
The agent tells you it worked
The agent tells you it worked

4 · CHECKING THE RESULT

Don't let the attacker grade itself

Checks run on the target and victim side, never in the attacker's context.



Define a computable state change beforehand and deterministically check it afterwards.

Example: a read-only database count before and after the exploit

4 · CHECKING THE RESULT

The report claimed a two-step attack

THE REPORT

An attacker controls the text of a Grafana tooltip. JavaScript in it runs in an admin's browser and invites another admin.

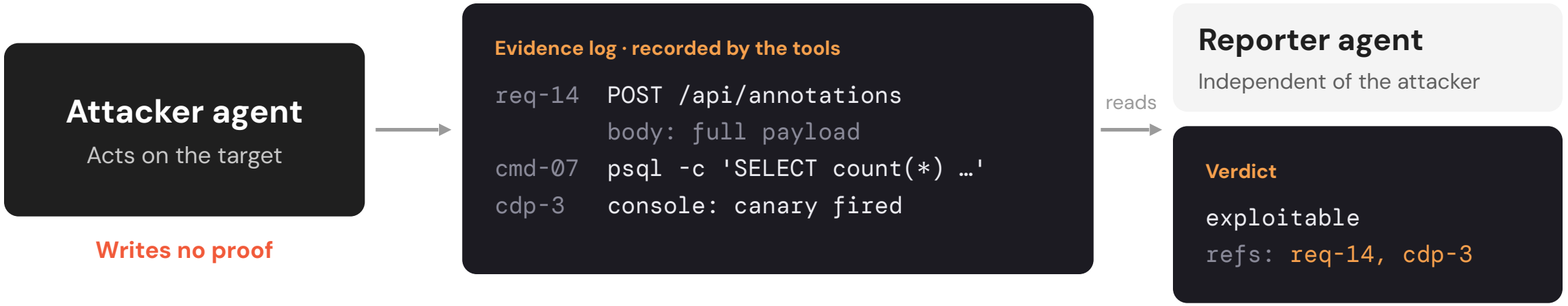
CLAIM	WHAT WE CHECKED	RESULT
1. Run JavaScript in the admin's browser	Browser request to our test URL	Confirmed
2. Send an invitation as that admin	No invitation payload tested	Unverified

Full attack chain: **INCONCLUSIVE**

4 · CHECKING THE RESULT

The proof is recorded, not written

Tools capture full payloads, commands and browser events. An independent reporter agent gives the verdict and points at them.



> The verdict **points at** recorded proof. No agent writes it.

Illustrative entries. A ref missing from the log sends the run back.

DEMO

Deleting across organizations

Grafana 10.2.5 · CVE-2024-1313

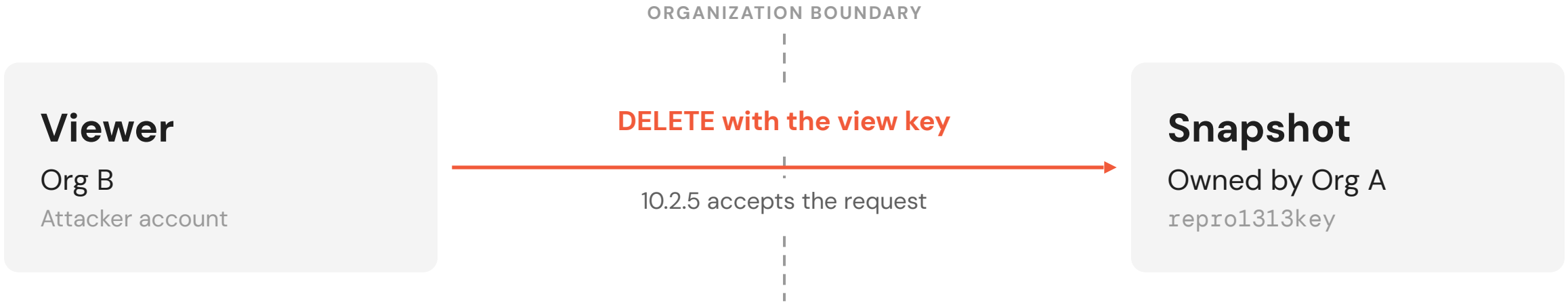
Saved exploit, live lab and verification.

5

5 · DEMO

A Viewer can delete another org's snapshot

CVE-2024-1313 · demo: 10.2.5 · vendor fix: 10.2.6




```
DELETE /api/snapshots/repro1313key
```

View key known. Delete key withheld. No membership in Org A.

5 · DEMO

The target confirmed the deletion

Recorded lab run · Grafana 10.2.5

BEFORE

1 Snapshot present



AFTER

0 Snapshot deleted

CONTROLLER QUERY · READ-ONLY

```
SELECT count(*) FROM dashboard_snapshot
WHERE "key" = 'repro1313key';
```

REPRODUCED Victim browser: “Snapshot not found”

The demo ran attacker agents on a live lab. Part 6: keeping that safe.

SAFETY

Reports are untrusted input

We feed text written by strangers to agents with a shell and credentials.

Report text is screened at submission, and any agent that detects an injection stops the run.

6

6 · SAFETY

Four safety patterns

1	**Treat the report as untrusted**	Any stage can stop the run on an injection.
2	**Isolate the lab**	Dedicated account, 1 h session, fresh VMs per run.
3	**Keep the attacker on the target**	Deny-by-default egress. Cloud metadata always blocked.
4	**Bound the run**	Turn caps, a $20 circuit breaker, a reaper for orphans.

> Agents flag. **The worker enforces.**

TRY IT

Reproduce the method yourself

An open-source CLI and lab format. Runs on a laptop with Docker and one API key.

7

7 · TRY IT

What we'd tell you before you build one

01

Budget for setup,

not for exploit writing.

02

Check the target,

not the agent's answer.

03

Record why runs fail.

That list is your roadmap.

04

Keep the pipeline model-agnostic.

Models change every few months.

Don't trust the report.

Don't trust the agent.

Trust the target.

REPRODUCING THE EXPLOIT, NOT THE REPORT

Questions?

Hedi Sfaxi · Konvu

QR to the repo


OWASP
GLOBAL AppSec
SF'26
November 2-6
THANK YOU!